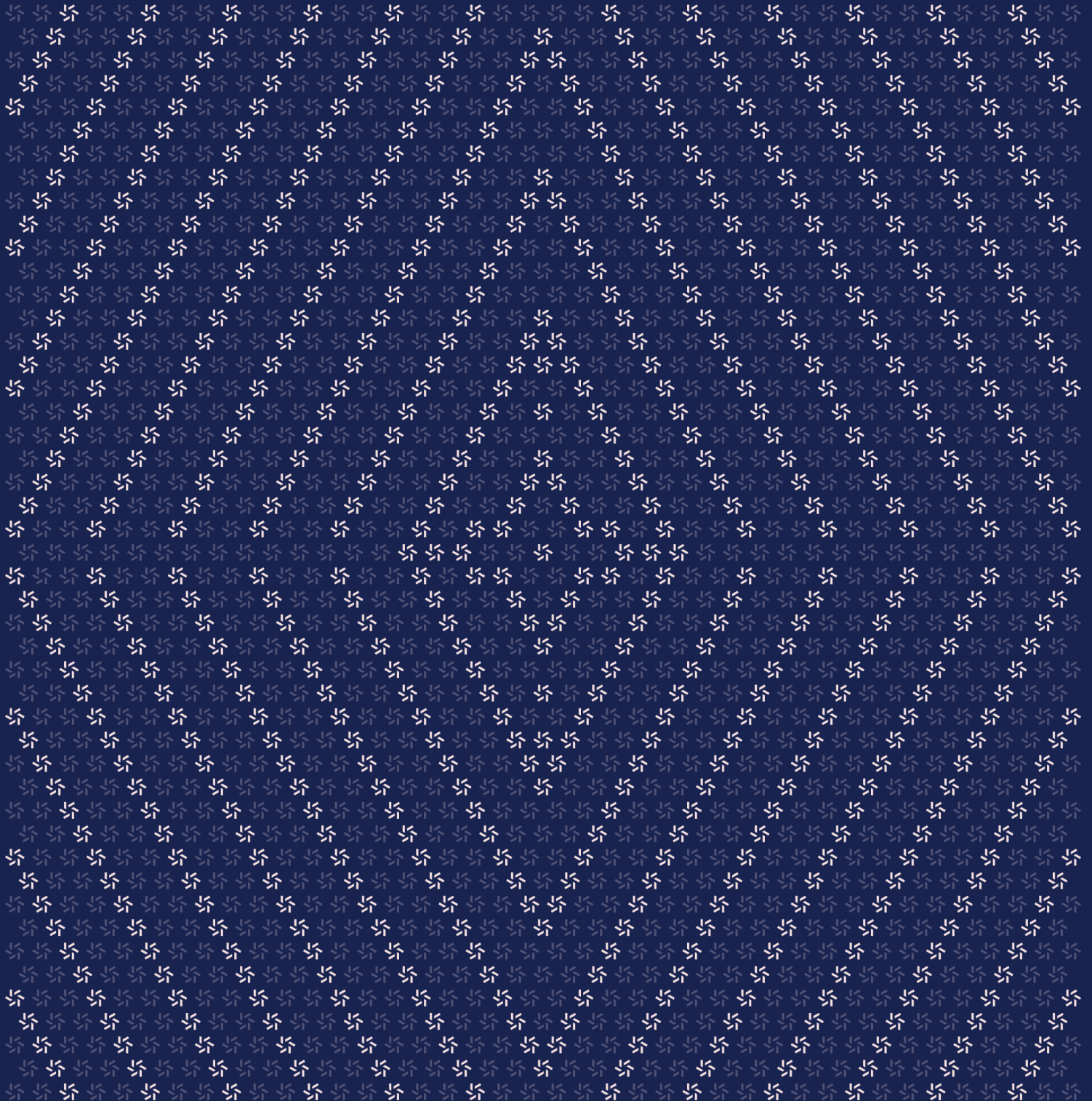


June 17, 2026

Locus Smart Contracts

Smart Contract Security Assessment



Contents

About Zellic	4
1. Overview	5
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	5
2. Introduction	7
2.1. About Locus Smart Contracts	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	9
2.5. Project Timeline	10
3. Detailed Findings	11
3.1. Permit2 authorization is replayable across all wallets sharing a KMS key or owner	11
3.2. Both execute and executeBatch can strand attached ETH	13
3.3. Uninitialized implementation contract	14
3.4. Both getAddress and isDeployed return addresses that createWallet will never deploy	15

4.	Discussion	16
4.1.	Revoking a permissioned key does not clear existing allowances	16
5.	System Design	17
5.1.	Component: LocusSmartWalletV2	17
5.2.	Component: Subwallet	19
5.3.	Component: LocusFactoryV2	20
5.4.	Component: LocusOwner7702	22
5.5.	Component: LocusPaymentRouter	23
6.	Assessment Results	25
6.1.	Disclaimer	25

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) [worldwide](#) in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io [and](#) follow [@zellic_io](#) [on Twitter](#). If you are interested in partnering with Zellic, contact us at hello@zellic.io [.](#)



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Locus from June 8th to June 11th, 2026. During this engagement, Zellic reviewed Locus's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Is the EIP-7702 contract secure?
 - Are the ERC-20 interactions in the smart wallet secure?
 - Is there any way for the subwallets to accidentally lose a user's money?
 - Can an unintended ownership change or abuse of a public function (e.g., by the permissioned key) lead to a loss of funds?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Off-chain infrastructure (backend services, bundler/paymaster, and the claim flow)
- Key custody and management for the KMS and owner keys

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

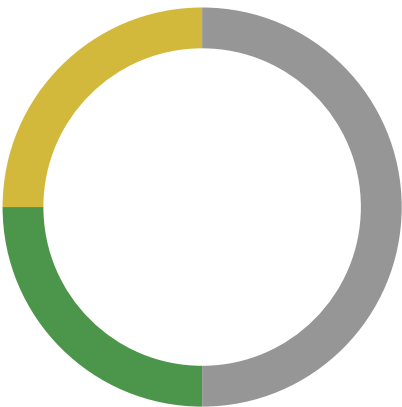
1.4. Results

During our assessment of the scoped Locus smart contracts, we discovered four findings. No critical issues were found. One finding was of medium impact, one was of low impact, and the remaining findings were informational.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Locus in the Discussion section ([4.7](#)).

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	1
Low	1
Informational	2



2. Introduction

2.1. About Locus Smart Contracts

Locus contributed the following description of the Locus smart contracts:

Locus is Base USDC payment infrastructure for agent-driven commerce, centered on ERC-4337 smart wallets with passkey ownership and permissioned backend spending. It supports direct sends, checkout routing, x402/MPP settlement, and time-boxed subwallet escrows for "send to anyone" flows.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case

basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped contracts itself. These observations — found in the Discussion ([4.7](#)) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

Locus Smart Contracts

Type	Solidity
Platform	EVM-compatible
Target	LocusContracts
Repository	https://github.com/locus-technologies/LocusContracts ↗
Version	e35abaee62a47c1d2b6520fcc02778b873dec8c7
Programs	LocusFactoryV2 LocusOwner7702 LocusSmartWalletV2 Subwallet LocusPaymentRouter

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 1.1 person-weeks. The assessment was conducted by two consultants over the course of four calendar days.

Contact Information

The following project manager was associated with the engagement:

Pedro Moura
✈ Engagement Manager
pedro@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Filipe Alves
✈ Engineer
filipe@zellic.io ↗

Weipeng Lai
✈ Engineer
weipeng.lai@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

June 8, 2026 Start of primary review period

June 11, 2026 End of primary review period

3. Detailed Findings

3.1. Permit2 authorization is replayable across all wallets sharing a KMS key or owner

Target	LocusSmartWalletV2		
Category	Business Logic	Severity	High
Likelihood	Low	Impact	Medium

Description

Permit2 settlement is one mechanism for pulling funds from LocusSmartWalletV2. During initialization, the wallet grants Permit2 an unlimited allowance for the `primaryToken`, allowing Permit2 to transfer that token from the wallet once it receives a valid authorization.

The ERC-1271 entry point used by Permit2 to validate those authorizations is the `isValidSignature` function in LocusSmartWalletV2. However, its validation logic does not bind the signed digest to the specific wallet contract. The KMS path recovers the signer from the raw digest, while the owner path forwards the same raw digest to the owner contract. Neither path commits to `address(this)`.

This creates a replay risk because Permit2's `SignatureTransfer` digest does not include the token owner. Permit2 signs over fields such as token, amount, spender, nonce, deadline, and witness, but the owner address is supplied separately during validation. Nonces are also tracked per owner, so replaying the same nonce against another wallet is not prevented.

As a result, if two wallets share the same KMS key or the same owner contract, a single Permit2 digest and signature accepted by wallet A will be accepted byte-for-byte by wallet B. Since both wallets have already approved Permit2 for their `primaryToken`, the same authorization can be replayed to pull funds from every wallet sharing that signer or owner.

Impact

A valid Permit2 authorization for one LocusSmartWalletV2 can be replayed against any other wallet that shares the same KMS key or owner contract. Since each wallet grants Permit2 an unlimited allowance for its `primaryToken`, the replayed authorization can be used to pull funds from every affected wallet.

Recommendations

Bind the ERC-1271 validation to `address(this)` for both signer paths so an authorization accepted by one wallet is rejected by every other.

Remediation

This issue has been acknowledged by Locus, and a fix was implemented in commit [a87a52fc](#).

3.2. Both execute and executeBatch can strand attached ETH

Target	LocusSmartWalletV2		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

LocusSmartWalletV2 is intended to be an ERC-20-only wallet that never holds ETH. To enforce this, both `receive` and `fallback` functions revert, and the internal `_call` helper rejects any nonzero per-call value.

This invariant is incomplete. Both `execute` and `executeBatch` are declared payable and validate only the `value` parameter passed to `_call`, never `msg.value`. As a result, a call such as `execute{value: x}(target, 0, data)` passes the `value == 0` check while the attached `x wei` is silently accepted and retained by the wallet.

Impact

If an authorized flow calls `execute` or `executeBatch` with attached ETH, the funds are accepted by the wallet but cannot be forwarded or withdrawn through any of its paths. The ETH therefore becomes permanently locked in the contract, breaking the intended no-ETH invariant.

Recommendations

Reject any attached ETH by adding an explicit `msg.value` check at the start of both `execute` and `executeBatch`:

```
if (msg.value != 0) revert EthNotSupported();
```

Remediation

This issue has been acknowledged by Locus, and a fix was implemented in commit [0f4c02be](#).

3.3. Uninitialized implementation contract

Target	Subwallet		
Category	Code Maturity	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

Subwallet is deployed once as a shared implementation (logic) contract and used through proxies — an EIP-1167 clone per subwallet. This implementation contract does not lock its initializer in a constructor, so `initialize` can be called directly on the implementation contract itself by any account.

Impact

This has no practical impact in the codebase at the time of writing: there is no `selfdestruct`, and `delegateExecute` and `_authorizeUpgrade` revert, so an attacker who initializes an implementation cannot reach the proxies or any privileged action. We raise it as a defense-in-depth and best-practice matter, as leaving the implementations initializable is a well-known antipattern that is trivial to eliminate.

Recommendations

Lock the initializer on Subwallet by invoking `_disableInitializers()` in the constructor, so the logic contracts can never be initialized directly.

Remediation

This issue has been acknowledged by Locus, and a fix was implemented in commit [657a9e81](#).

3.4. Both getAddress and isDeployed return addresses that createWallet will never deploy

Target	LocusFactoryV2		
Category	Coding Mistakes	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The createWallet function in LocusFactoryV2 rejects deployments whose owner7702 or initialPermissionedKey is the zero address, reverting with InvalidOwner or InvalidPermissionedKey. The prediction helpers getAddress and isDeployed, however, derive and return a counterfactual address for any input tuple without applying those checks.

As a result, the factory can advertise a deterministic address for a tuple that createWallet will always reject, so no account can ever be deployed at — or take control of — that address.

Impact

The impact is limited. Funding an address derived from a zero owner7702 or initialPermissionedKey serves no legitimate purpose, so this is not reachable in normal operation. The realistic concern is an integration that mistakenly passes address(0) and funds the predicted address; any ERC-20 tokens sent there would be permanently stuck, since the factory has no path to deploy a wallet at that address. This is primarily a defense-in-depth and consistency consideration rather than an exploitable issue.

Recommendations

Mirror createWallet's validation in getAddress and isDeployed: revert when owner7702 or initialPermissionedKey is the zero address, so the prediction helpers never return an address the factory cannot deploy.

Remediation

This issue has been acknowledged by Locus, and a fix was implemented in commit [20ed2259](#).

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Revoking a permissioned key does not clear existing allowances

A permissioned (KMS) key can set ERC-20 or Permit2 allowances on the wallet's behalf through `execute`. While the `revokePermissionedKey` function in `LocusSmartWalletV2` deactivates the key in the wallet's signer state, it does not reach allowances the key already set — these live in the token and are spent through `transferFrom`, which needs no signature from the wallet. So a key that set `approve(attacker, max)` before being revoked can keep draining deposits that arrive after the revocation.

We do not consider clearing prior approvals to be `revokePermissionedKey`'s responsibility, and enumerating them on chain is impractical; the same applies to `transferWalletOwnership`. We raise it as a documentation matter: the documentation presents `revokePermissionedKey` as the recovery mechanism for a compromised key, but a revoke alone does not fully restore safety. Owners should be aware that full recovery means migrating to a fresh wallet address, which carries no preexisting allowances.

5. System Design

This provides a description of the high-level components of the system and how they interact, including details like a function's externally controllable inputs and how an attacker could leverage each input to cause harm or which invariants or constraints of the system are critical and must always be upheld.

Not all components in the audit scope may have been modeled. The absence of a component in this section does not necessarily suggest that it is safe.

5.1. Component: LocusSmartWalletV2

Description

LocusSmartWalletV2 is the user's account: a general-purpose, ERC-20-only ERC-4337 smart wallet, deployed as a per-user ERC-1967 proxy of a shared implementation. Every action is authorized by one of two signer classes — the ERC-1271 owner (in production, an EIP-7702/passkey account) or a KMS permissioned key held by the Locus backend.

It is responsible for the following:

- **Execution.** Both `execute` / `executeBatch` make external-only (`target != address(this)`), zero-value calls and bubble revert data.
- **Two-signer authorization.** The `[signerHint | kind | payload]` envelope is parsed by `_validateSignature`, which also checks the `UserOp`'s outer selector against a per-class, default-deny allowlist.
- **Subwallets.** The following deploys, recycles, and drains single-token earmarking subaccounts: `createAndFundSubwallet` / `disburseSubwallet` / `reclaimSubwallet`.
- **The admin (owner only).** The owner alone may call `setPermissionedKey` / `revokePermissionedKey` / `transferWalletOwnership`.
- **Off-chain signatures.** Permit2, EIP-3009/x402, and MPP pulls are authorized by `isValidSignature`.
- **Permit2.** The `primaryToken` is granted an unbounded Permit2 allowance at initialization (skipped when zero).

The inherited Solady surface is disabled: `storageStore`, `delegateExecute`, `addDeposit` / `withdrawDepositTo`, the ownership-handover trio, `renounceOwnership`, `transferOwnership`, and `_authorizeUpgrade` revert, and `receive` / `fallback` reject ETH. The `EntryPoint` is pinned to V0.9.

Invariants

- The KMS class is allowed `execute`, `executeBatch`, and the three subwallet selectors; the owner adds the admin trio. The allowlist is default-deny.
- A KMS operation can never reach an admin function: `execute` / `executeBatch` forbid `target == address(this)`, and there is no self-call path.
- The `activePermissionedKey` is the single source of truth — every accepted KMS signature (in both validation paths) must recover to it and be active, so a `revoke` or

rotation invalidates prior KMS signatures at once.

- No ETH enters through intended paths; `execute / executeBatch` reject a nonzero value argument, and `receive / fallback / addDeposit / withdrawDepositTo` revert.
- The `owner()` is never zero, the contract is nonupgradable, and `execute / executeBatch` and subwallet management are non-reentrant.

Test coverage

Cases covered

- Both `execute / executeBatch` — the following are covered: happy paths, self-target and nonzero-value rejection, `onlyEntryPointOrOwner` gating, revert bubbling/rollback, and the disabled surfaces (`delegateExecute`, `addDeposit`, `withdrawDepositTo`, `receive`, NFT callbacks) reverting.
- Selector allowlist — KMS passes for its set and fails for the admin trio; the owner passes for the trio and `execute`; and unknown, short, and envelope-only selectors, incorrect hint, and inactive/rotated-out keys fail.
- Admin — owner happy paths and reverts (zero key, revoke-with-none-active, transfer-to-zero) are covered; the KMS class cannot self-rotate, directly revoke, or transfer ownership.
- The `isValidSignature` function — KMS accept/reject (incorrect hash, non-KMS, postrevoke, postrotation) and owner forwarding (reject/revert, non-65-byte, empty, and oversized fall-through) are covered.
- Off-chain signatures and flows — x402 / EIP-3009 wrapped KMS and owner signatures, including postrevoke rejection; initialization and Permit2 approval (with the zero-token skip); and end-to-end `handleOps` flows are covered.
- Invariants — the following are covered: token conservation, active-key consistency, owner-never-zero, subwallet cap/consistency, and "KMS is never validated for an admin selector".

Cases not covered

- ETH attached as `msg.value` to `execute / executeBatch` (as opposed to the value argument) is not covered — the gap behind the corresponding finding (section [3.2](#), [7](#)).
- The bookkeeping desync from invoking `Subwallet.transferToken / activate` directly through `execute` is not covered.

Attack surface

- **`execute / executeBatch`.** The caller controls target, the value argument, and data; access is gated to the `EntryPoint` and the owner, and the KMS class reaches them only via an allowlisted `UserOp`. Self-target and nonzero value are rejected, closing the admin and ETH paths; target / data are otherwise arbitrary, which defines a leaked KMS key's blast radius (drain any ERC-20 and make arbitrary calls, but never reach the admin trio). The unchecked `msg.value` is the one input that escapes the no-ETH

invariant.

- **_validateSignature.** The attacker controls the signature and calldata; the signer hint must equal the active key or owner, a KMS signature must recover via `ecrecover` to the active key, and the outer selector must be allowlisted. It fails closed; the owner path `staticcalls` the owner contract (which bundlers reject).
- **isValidSignature.** The attacker controls the hash and signature; the KMS path requires the recovered signer to equal the active key, and the owner path forwards via `try/catch`. The digest is not bound to `address(this)`, enabling cross-account replay when a key or owner is shared (see the corresponding finding, section 3.1.7).
- **Subwallet management.** The caller controls the token, amount, subwallet, and recipient; `disburseSubwallet` rejects a zero or self recipient. Draining a subwallet directly through `execute` bypasses the active-set bookkeeping, which is self-healing.
- **Admin trio.** This is owner-only and reachable only as a top-level owner-allowed selector or a direct owner call.

5.2. Component: Subwallet

Description

Subwallet is a minimal single-ERC-20 earmarking subaccount, deployed as an EIP-1167 clone by a main wallet to isolate a pending spend (for example, a send-to-anyone-by-email transfer) in its own watchable address. It holds one token and can move it to any nonzero address at any time on the authority of the main wallet or the main wallet's live owner. It has no fixed recipient and no timelock — it is not a trustless escrow, and payout correctness is enforced off chain by the claim flow.

It is responsible for the following:

- **Escrow of one token.** Both `initialize` / `activate` set the token; only one is held at a time.
- **Release.** The token is sent to an arbitrary nonzero recipient (main wallet or live owner) by `transferToken`.
- **Recycle bookkeeping.** Both `activate` / `deactivate` are main-wallet-only, keeping the parent's active-set and recycle pool in sync.
- **The live owner.** The `owner()` is fetched from the main wallet on every check, never cached.

Invariants

- A subwallet is initialized once, and its token must have code.
- Both `activate` / `deactivate` are main-wallet-only; `transferToken` is restricted to the main wallet or the live owner.
- The owner is always read live, so an ownership change propagates immediately and a prior owner cannot reach existing subwallets.
- The `transferToken` function rejects a zero recipient and uses a tolerant ERC-20

transfer; the subwallet holds no ETH.

- Reads against the main wallet revert on a malformed response, surfacing integration faults loudly.

Test coverage

Cases covered

- The following cases are covered: `initialize` (state, one-shot guard, no-code token); `transferToken` by the main wallet and owner to arbitrary addresses, with rejection for a random caller, a zero recipient, and a failed or `false`-returning transfer, plus the owner-`staticcall`-failure path; `getTokenBalance`, `isActive`, and `owner` (including reverts when the main wallet reverts or is an EOA); `activate` (main only, no-code token, token switch); `receive` reverting; and invariants (owner equals main owner, balance/active consistency).

Cases not covered

- The token-stranding path — `activate` to a new token (through the main wallet's `execute`) while a prior balance is held — a recoverable bookkeeping consequence rather than a loss.

Attack surface

- **transferToken.** The caller controls the recipient and amount; access is restricted to the main wallet or live owner, a zero recipient is rejected, and the recipient is otherwise unconstrained. The subwallet gives no on-chain payout guarantee — correctness is off chain — and because ownership transfers take effect instantly, a former owner immediately loses access.
- **activate / deactivate.** These are main-wallet only but reachable via the main wallet's `execute`, which can strand a residual balance until the subwallet is reactivated to the old token (recoverable).
- **Live main-wallet reads.** Every authorization decision `staticcalls` the main wallet, so the subwallet's safety depends on a well-formed parent.

5.3. Component: LocusFactoryV2

Description

LocusFactoryV2 is a permissionless, deterministic CREATE2 factory that mints LocusSmartWalletV2 instances as ERC-1967 proxies of a single immutable implementation. It holds immutable references to the wallet and subwallet implementations and an optional `primaryToken` (zero for a token-agnostic wallet).

It is responsible for the following:

- **Deploying and initializing.** Via `CREATE2`, `createWallet(owner7702, salt, initialPermissionedKey)` deploys a proxy and initializes it in one transaction; it is idempotent.
- **Predicting addresses.** Both `getAddress / isDeployed` compute and check the counterfactual address before deployment.
- **Binding the address.** The `CREATE2` salt is `keccak256(owner7702, salt, initialPermissionedKey)`.

Invariants

- The salt commits to all three inputs, so changing any one yields a different address, binding each wallet address to its exact owner and KMS key.
- The `createWallet` function is idempotent (an already deployed address is returned without reinitializing) and rejects a zero owner, a zero key, and attached ETH.
- The constructor rejects a zero or no-code implementation or subwallet implementation and a nonzero no-code `primaryToken`; a zero `primaryToken` is allowed.

Test coverage

Cases covered

- The following cases are covered: `createWallet` (initialization, predicted-address match, zero-owner/zero-key reverts, idempotent repeat-create, zero-`primaryToken` wallet); constructor reverts (zero/no-code implementation, subwallet implementation, no-code `primaryToken`) and the zero-`primaryToken` allowance; `isDeployed` before and after; the salt high-bits collision case; and fuzzing (determinism, different-KMS-different-address, idempotency, zero-token wallets).

Cases not covered

- Both `getAddress / isDeployed` do not enforce the zero-owner/zero-key rejection that `createWallet` applies, so they can return an undeployable address — the gap behind the corresponding finding (see section [3.4](#), ↗).
- Rejection of attached ETH on `createWallet` is not directly exercised.

Attack surface

- **`createWallet`.** This is permissionless; the caller controls `owner7702`, `salt`, `initialPermissionedKey`, and `msg.value`. A zero owner or key and attached ETH are rejected. Because the salt binds the address to the owner and KMS key, the only wallet anyone can deploy at a given address is the victim's own correctly configured wallet, which is harmless.
- **`getAddress / isDeployed`.** These are unvalidated views, so they return a deterministic address even for a tuple `createWallet` will reject; funding such an address would

strand the funds.

- **Constructor.** These are deployer-controlled immutables, each validated for code presence.

5.4. Component: LocusOwner7702

Description

LocusOwner7702 is the EIP-7702 delegate that lets a user's EOA serve as the WebAuthn-passkey-controlled ERC-1271 owner of a LocusSmartWalletV2. With this code installed via EIP-7702, `address(this)` is the owner EOA, which validates P-256 passkey assertions and relays passkey-authorized calls to the wallet; the EOA key stays out of normal use.

It is responsible for the following:

- **Onboarding a passkey.** A passkey is registered through `initializePasskey` (an EOA self-call) or `setPasskeyWithEoaSignature` (a relayer submits while the EOA signs the data).
- **Rotating.** The current passkey authorizes its own replacement through `rotatePasskey`.
- **Executing as the owner.** A call to `execute` runs a passkey-authorized call in which the target sees `msg.sender == address(this)` — the path for relaying owner/admin actions to the wallet.
- **Upholding ERC-1271.** Through `isValidSignature`, the delegate verifies a WebAuthn assertion over an account- and chain-bound challenge.

Invariants

- Challenges are domain-separated by `address(this)` and `block.chainid` (plus a per-operation typehash).
- A monotonic nonce guards every passkey-authorized state change and execution.
- A passkey must be initialized before `rotatePasskey` / `execute` / `isValidSignature` succeed, and its coordinates are never zero.
- The `initializePasskey` function is a one-shot self-call; `setPasskeyWithEoaSignature` needs a nonexpired deadline and an ECDSA signature recovering to the EOA; `rotatePasskey` and `execute` need a nonexpired deadline and a valid current-passkey assertion.
- Allowing the EOA to set the passkey by signature does not expand authority, since the EOA can redelegate regardless.

Test coverage

Cases covered

- The following cases are covered: onboarding (self-call and EOA-signature); `isValidSignature` accepting an assertion and the challenge being owner- and chain-bound; `execute` consuming the nonce, calling as the owner, and reaching a wallet admin function; `rotatePasskey` updating the key and nonce; the wallet accepting an Owner7702-wrapped signature; and the reverts. These are `initializePasskey` (not-self, already initialized, zero key), `setPasskeyWithEoaSignature` (expired, bad signer), `rotatePasskey` (uninitialized, expired, invalid, zero key), `execute` (uninitialized, expired, invalid, target reverts), and `isValidSignature` (uninitialized and verification failure).

Cases not covered

- An end-to-end replay of a captured assertion against a different account or chain is not exercised directly (only the challenge binding is asserted), and the compact-versus-full WebAuthn decode branches are exercised only indirectly.

Attack surface

- **`isValidSignature`.** The attacker controls the hash and the blob; acceptance needs a WebAuthn assertion over the account/chain/hash-bound challenge against the stored passkey. Replay protection for the underlying hash still lives in that hash.
- **`execute`.** The attacker controls target, value, data, deadline, and the authorization; a valid assertion over (target, value, dataHash, nonce, deadline) is required, bounded by the nonce and deadline. It runs as the owner EOA, so it can reach the wallet's owner-only functions — security reduces to control of the passkey.
- **`setPasskeyWithEoaSignature`.** It requires an EOA ECDSA signature over the exact digest — no authority expansion beyond the EOA key.
- **`rotatePasskey`.** It requires a current-passkey assertion, bounded by the nonce and deadline.

5.5. Component: LocusPaymentRouter

Description

LocusPaymentRouter is a standalone, stateless USDC router for Checkout sessions — not a wallet, with no proxy, no admin, and an immutable USDC address — and it never holds funds. It moves one USDC payment directly from payer to seller and emits a `sessionId`-tagged event for off-chain reconciliation. It is independent of the smart wallet; the payer can be any USDC holder.

It is responsible for the following:

- **Routing.** USDC is transferred from the preapproved payer to the recipient by `pay(sessionId, recipient, amount)`.
- **Single-transaction payment.** First, `payWithPermit` applies an EIP-2612 permit, then transfers.
- **Tagging.** Each payment emits `CheckoutPayment(sessionId, from, to, amount)`.

Invariants

- The contract never holds funds; USDC moves payer-to-recipient directly.
- Inputs are validated — a nonzero `sessionId` and amount as well as a recipient that is nonzero and not the router or USDC.
- The USDC address is immutable and nonzero.
- The permit in `payWithPermit` is best-effort (`try/catch`), so a failed or redundant permit does not block the payment; the `safeTransferFrom` is the authoritative settlement.
- Matching is off chain via the `sessionId`; no on-chain session state exists to grief.

Test coverage

Cases covered

- The following cases are covered: constructor (sets USDC, reverts on zero); `pay` (happy path and reverts for a zero `sessionId`, a zero/router/USDC recipient, and a zero amount); `payWithPermit` (happy path, the `try/catch` fallback on an existing allowance, and validation reverts).

Cases not covered

- Nonstandard or fee-on-transfer USDC variants are out of scope; USDC on Base is assumed standard.

Attack surface

- **pay / payWithPermit.** These are permissionless; the caller controls `sessionId`, recipient, amount, and the permit parameters. Funds always move from `msg.sender`, so a caller spends only their own approved USDC; the recipient checks reject the router and USDC, and `sessionId / amount` must be nonzero. The permit is best-effort and the transfer is authoritative, so a failed permit produces no unfunded event.
- **Off-chain matching.** The backend reconciles and dedupes by `sessionId`; with direct payer-to-seller transfers and no on-chain state, there is nothing to grief.

6. Assessment Results

During our assessment on the scoped Locus smart contracts, we discovered four findings. No critical issues were found. One finding was of medium impact, one was of low impact, and the remaining findings were informational in nature.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.